KEYWORD · SEMANA 7 · INFRAESTRUTURA DE INFERÊNCIA 2/2

KV cache e batching

Como um servidor atende muita gente com a mesma GPU: a memória de cada pedido, e a agenda que decide quem roda em cada passo.

KV cache | PagedAttention | batching contínuo | chunked prefill | prefix caching

BA Beatriz Almeida · 30/10/2026

A PERGUNTA

20 pedidos ao mesmo tempo. Quanto custa?

Medi no meu notebook com o batching.py: quanto tempo o modelo leva para escrever 1 token para cada pedido.

1 PEDIDO

26 ms

para escrever 1 token

SE CUSTASSE 20×

526 ms

20 × 26 ms: um pedido depois do outro

MEDIDO: 20 JUNTOS

69 ms

1 token para cada um dos 20, no mesmo passo

Era para levar **526 ms**. Levou **69 ms**: 13% do esperado.

Por que servir 20 pedidos ao mesmo tempo custa muito menos que 20× servir um?

ROTEIRO

Cinco peças, uma pergunta

A primeira explica por que o lote é quase de graça. As outras quatro são como os servidores fazem caber mais pedidos no lote sem travar ninguém.

1

Por que o lote é barato

O decode passa o tempo esperando a memória. O lote aproveita essa espera.

2

KV cache como memória

Cada pedido ocupa memória com as suas anotações. É isso que limita o lote.

3

PagedAttention

Guardar as anotações em blocos pequenos, sem reservar espaço à toa.

4

Contínuo e chunked prefill

Quem termina sai, quem chega entra, a cada passo. Prompt longo vai em pedaços.

5

Prefix caching

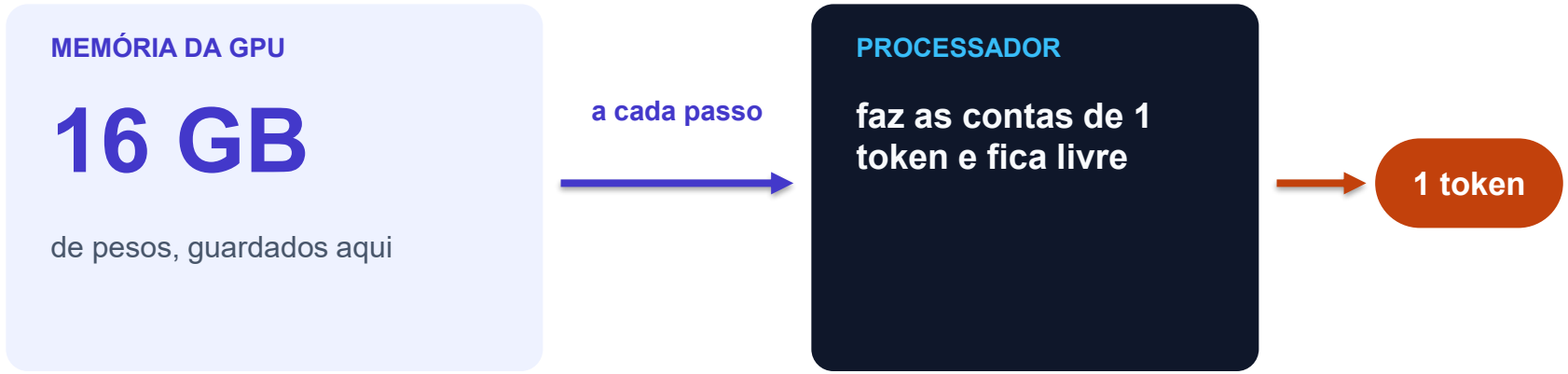
Começo igual, anotações iguais: fazer uma vez e emprestar.

Duas perguntas guiam tudo: quanto tempo cada passo leva? E quantos pedidos cabem na memória?

1 · POR QUE O LOTE É BARATO

No decode, a GPU passa quase todo o tempo esperando

Exemplo: Llama-3.1-8B numa RTX 4090. Para escrever UM token, o modelo inteiro precisa sair da memória e passar pelo processador.



TRAZER OS PESOS: 34 MS

Por passo, **~15 GB** atravessam a memória. A 4090 entrega **~440 GB/s** na prática: 15 ÷ 440 = 0,034 s.

(O arquivo tem 16 GB, mas a tabela de entrada só é consultada numa linha por token.)

FAZER AS CONTAS: 0,2 MS

Cada peso faz 1 multiplicação e 1 soma: **2 × 8 bilhões = 16 bilhões** de contas. A 4090 faz ~80 trilhões por segundo.

A GPU espera 34 ms e trabalha 0,2 ms.

1 · POR QUE O LOTE É BARATO

A ideia: trazer os pesos uma vez e usar para 20

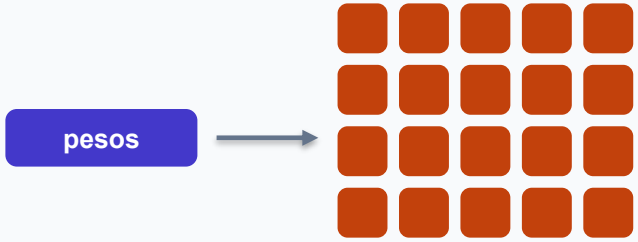
Se a GPU está parada esperando os pesos chegarem, que pelo menos cada viagem sirva para vários pedidos.

UM DE CADA VEZ



20 viagens
uma por pedido
(4 de 20 desenhadas)

TODOS NO MESMO PASSO



1 viagem
para os 20 tokens

UM DE CADA VEZ	
20 pedidos × 34 ms de viagem	**680 ms**

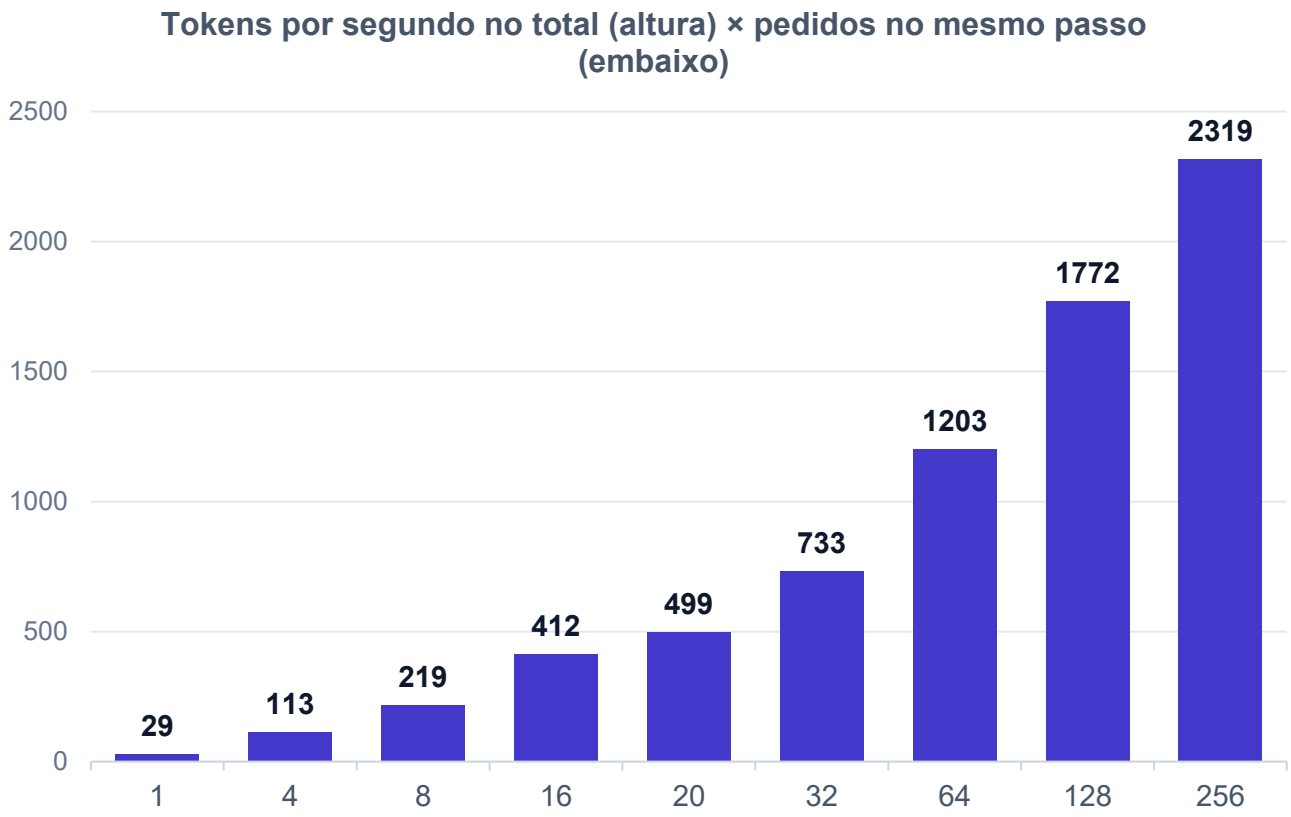
TODOS JUNTOS	
pesos 1× (34) + anotações dos 20 (6)	**40 ms**

Como um ônibus: a viagem custa quase o mesmo com 1 ou com 20 passageiros. As contas dos 20 (4 ms) acontecem enquanto a memória ainda está entregando.

1 · POR QUE O LOTE É BARATO

Quanto o servidor entrega com 1, 20 ou 256 pedidos

Conta do laboratório para o Llama-3.1-8B numa RTX 4090. Cada pedido já tem 1.000 tokens de conversa.



COMO LER

Cada barra é um tamanho de lote. A altura é quantos tokens o servidor entrega por segundo, **somando todos os pedidos**.

1 pedido: 29 tok/s. 20 pedidos: 499 tok/s, 17× mais, com a mesma GPU.

Repare: até 32 a barra quase dobra quando o lote dobra. Depois cresce menos, porque as anotações de cada pedido também precisam ser lidas.

Por que está aqui: o lote multiplica o que um servidor entrega sem comprar outra GPU.

1 · POR QUE O LOTE É BARATO

O preço do lote: cada pessoa fica um pouco mais lenta

Mesma conta do slide anterior, olhando para dois lados: quem paga o servidor e quem espera a resposta.

Pedidos no lote	Tok/s no total	Tok/s de cada pessoa	Custo de cada token
1	29	29,1	100%
8	219	27,4	13%
20	**499**	**25,0**	**5,8%**
64	1.203	18,8	2,4%
256	2.319	9,1	1,3%

LEIA ASSIM

Total: o que interessa a quem paga.

Cada pessoa: a velocidade com que o texto aparece na tela.

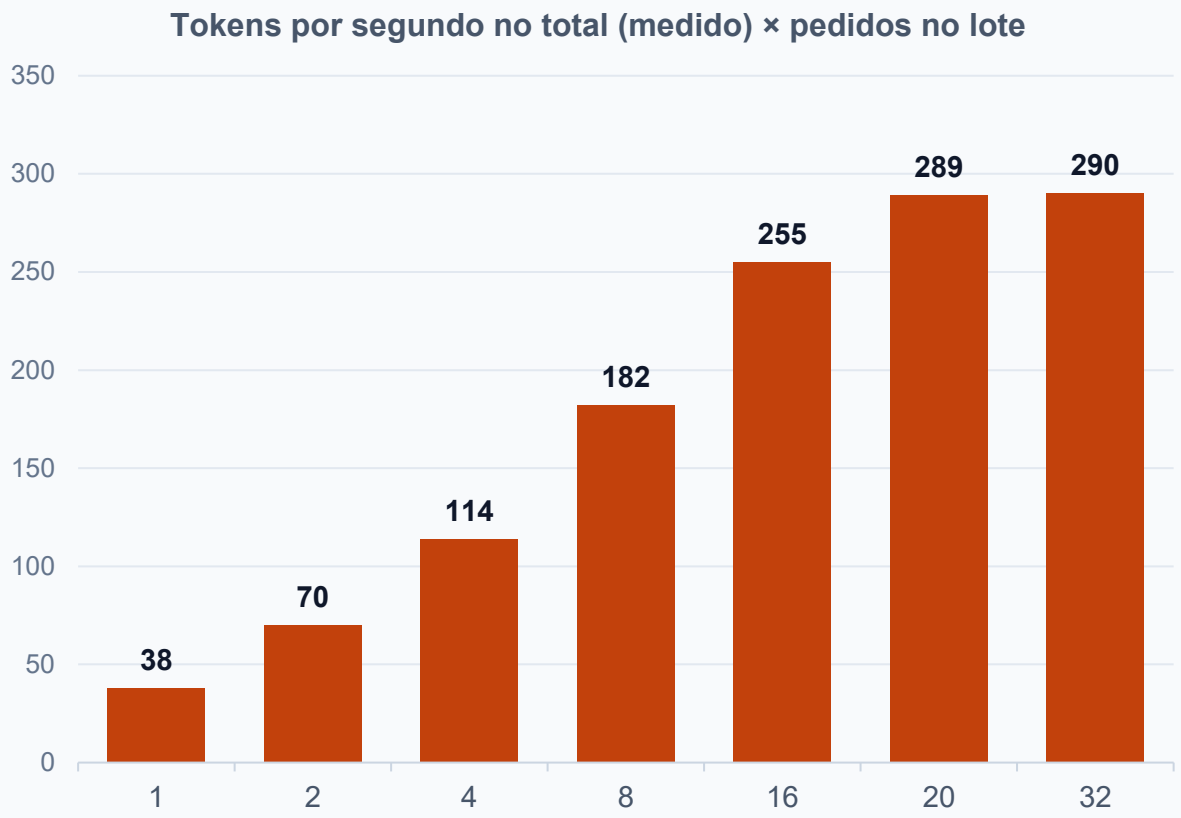
Custo: tempo de GPU por token, perto do lote de 1.

Com 20 pedidos, cada token custa 6% e cada pessoa ainda vê 25 tok/s, bem acima dos ~5 tok/s que alguém consegue ler. Com 256, o servidor rende mais, mas cada um cai para 9 tok/s.

1 · POR QUE O LOTE É BARATO

Medido: 20 juntos custam 13% de 20 separados

batching.py no meu notebook: modelo de teste de 510 MB, PyTorch na CPU (i5-1135G7), 256 tokens por pedido.



COMO LER

Mesma ideia do gráfico da GPU, mas medida de verdade, numa máquina que todo mundo tem.

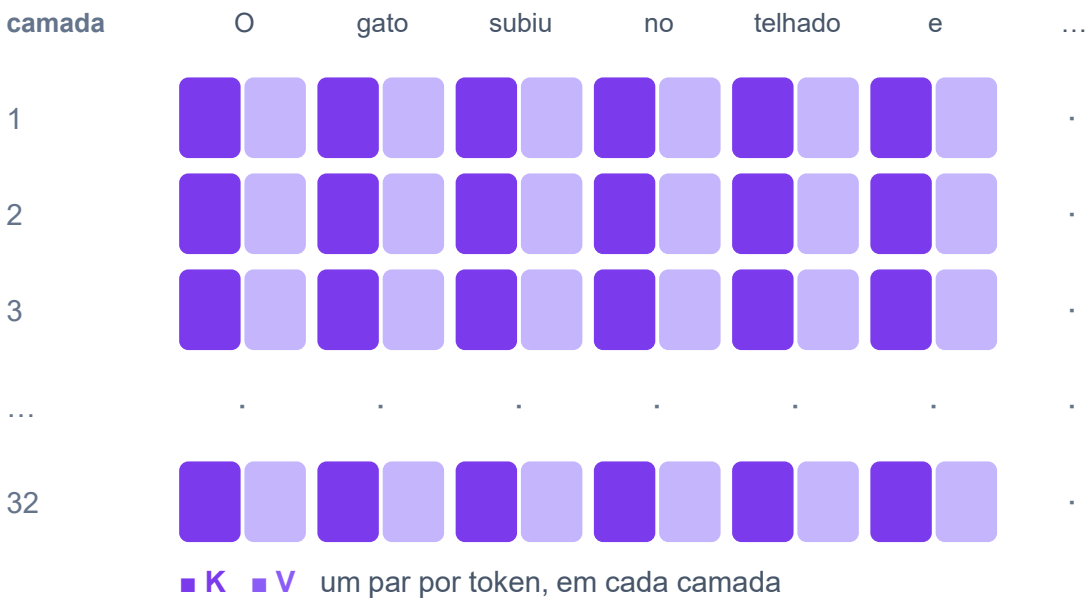
De 1 a 20 pedidos, o total vai de 38 para 289 tok/s: **7,6× mais barato** do que atender um por vez.

Depois de 20, estaciona (~290). A CPU faz poucas contas por segundo: deixou de esperar a memória e passou a esperar as contas. A GPU faz 250× mais contas, por isso lá o ganho vai mais longe.

Preço: cada pedido ficou 2,6× mais lento (38 → 14 tok/s).

Os pesos são de todos. O KV cache é de cada um

Lembrete da semana 6: para cada token lido, em cada camada, o modelo guarda dois vetores, K e V, para não precisar reler a conversa inteira.



EM PALAVRAS SIMPLES

K (chave): uma etiqueta que diz do que o token trata, para ser encontrado.

V (valor): o conteúdo que o token passa adiante quando é encontrado.

Para escrever o próximo token, o modelo compara a pergunta dele com todas as etiquetas e mistura os conteúdos.

ANALOGIA

Fichas de leitura: para cada palavra lida, uma ficha por camada. Ninguém relê o livro, mas as fichas ocupam a mesa.

O lote divide os pesos. As fichas não se dividem: 20 pedidos = 20 pilhas de fichas.

Quanto pesa: a conta, passo a passo

Os números vêm do arquivo config.json de cada modelo, que é público no Hugging Face.

LLAMA-3.1-8B, POR TOKEN

K e V: dois vetores	**2**
× camadas do modelo	**× 32**
× cabeças de K/V em cada camada	**× 8**
× números em cada vetor	**× 128**
× bytes por número (FP16)	**× 2**
= por token	**128 KB**
× 4.096 tokens (uma conversa longa)	**512 MiB**

Para comparar: uma conversa de 4.096 tokens ocupa 512 MiB, o mesmo que o modelo Qwen2.5-0.5B inteiro em FP16 ocupa só de pesos.

OUTROS MODELOS

modelo	por token	4.096 tokens
Qwen2.5-0.5B	12 KB	48 MiB
Llama-3.2-1B	32 KB	128 MiB
Qwen2.5-3B	36 KB	144 MiB
Llama-3.1-8B	**128 KB**	**512 MiB**

POR QUE SÓ 8 CABEÇAS DE K/V?

O Llama-8B tem 32 cabeças de atenção, mas elas dividem 8 pares de K/V (GQA). Sem isso, o cache seria 4× maior.

2 · KV CACHE COMO MEMÓRIA

Numa GPU de 24 GB, cabem só 10 conversas longas

Llama-3.1-8B em FP16 numa RTX 4090. A pergunta do dono do servidor: depois dos pesos, quantas pilhas de fichas cabem?



21,6 GB que o servidor pode usar

A CONTA

Memória da GPU	**24 GB**
× 90% (o resto fica para o sistema e contas temporárias)	**21,6 GB**
− pesos do modelo	**− 16,1 GB**
= sobra para o KV cache	**5,5 GB**
÷ 512 MiB por conversa de 4.096 tokens	**10 conversas**

POR QUE ISSO IMPORTA

No slide do gráfico, o lote ainda rendia com **mais de 100 pedidos**. A memória deixa só **10**.

Quem limita o lote não são as contas, é a memória do KV cache. Por isso as próximas três técnicas são sobre usar melhor essa memória.

O jeito antigo: reservar o pior caso para cada pedido

O servidor não sabe quando a resposta vai acabar. Então reservava, para cada pedido, espaço para 4.096 tokens, num pedaço só.



QUANTO É ÚTIL

usados: 1.000+300+2.600+150	**4.050**
reservados: 4 × 4.096	**16.384**
4.050 ÷ 16.384	**25%**

ANALOGIA

Um restaurante que reserva uma mesa de 40 lugares para cada cliente, porque "vai que chegam amigos".

NÃO É SÓ NESTE EXEMPLO

O artigo do vLLM mediu: nos servidores da época, **só 20% a 38%** da memória de KV guardava token de verdade.

Guardar as fichas em blocos pequenos e soltos

Em vez de uma reserva grande, blocos de 16 tokens. Quando um bloco enche, o pedido ganha outro, de qualquer lugar da memória.

1 · O QUE O PEDIDO VÊ

- A · bloco 0
- A · bloco 1
- A · bloco 2
- B · bloco 0
- B · bloco 1

2 · O ÍNDICE (TABELA)

A·0	→	bloco **7**
A·1	→	bloco **1**
A·2	→	bloco **12**
B·0	→	bloco **4**
B·1	→	bloco **9**

3 · ONDE ESTÁ NA MEMÓRIA DA GPU

0 · livre	1 · A1	2 · livre	3 · livre
4 · B0	5 · livre	6 · livre	7 · A0
8 · livre	9 · B1	10 · livre	11 · livre
12 · A2	13 · livre	14 · livre	15 · livre

COMO LER

O pedido A acha que tem 3 blocos em fila (0, 1, 2). O índice diz que eles estão nos blocos 7, 1 e 12 da GPU.

ANALOGIA

Fichário de folhas soltas: pega uma folha quando a anterior enche, e o índice guarda a ordem.

Resultado: de 10 para 41 conversas na mesma GPU

Só o último bloco de cada pedido pode ter lugar vazio. O exemplo usa blocos de 4 tokens para caber no slide.

BLOCO 0 → 7	BLOCO 1 → 1	BLOCO 2 → 12
O	telhado	Ele
gato	e	*vazio*
subiu	dormiu	*vazio*
no	.	*vazio*

9 tokens → 3 blocos. Só 3 lugares vazios, todos no último bloco.

A MESMA GPU DE 24 GB

Conversa típica: 1.000 tokens ÷ 16	**63 blocos**
63 × 16 = 1.008 lugares × 128 KB	**126 MiB**
Antes, cada conversa reservava	**512 MiB**
5,5 GB livres ÷ 126 MiB	**41 conversas**

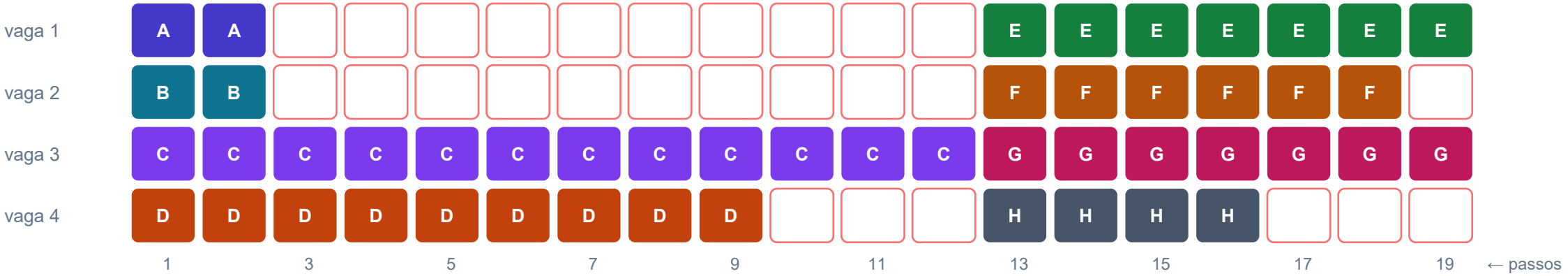
< 4% da memória de KV desperdiçada, medido no artigo (antes: 60% a 80%)

2 a 4× mais tokens por segundo que os servidores da época, porque cabem mais pedidos no lote

Bônus: como é tudo índice, dois pedidos podem apontar para o mesmo bloco. Isso vira o prefix caching (parte 5).

Batching estático: o lote espera o mais longo

8 pedidos na fila, 4 vagas no lote. Quantos tokens cada resposta tem: A=2 B=2 C=12 D=9 E=7 F=6 G=7 H=4.



COMO LER A GRADE

Linha = uma vaga do lote (um lugar no ônibus). **Coluna** = um passo: cada vaga ganha 1 token. **Letra** = o pedido sentado ali.

□ Borda vermelha: vaga presa, o pedido já acabou mas o lote não.

OS NÚMEROS

Passos até atender os 8	**19**
Casas com trabalho: 49 de 19 × 4 = 76	**64%**
Passo médio em que cada um recebe a resposta	**15,5**

4 · BATCHING CONTÍNUO

Batching contínuo: quem termina sai, quem espera entra

Os mesmos 8 pedidos e as mesmas 4 vagas. Agora o lote é remontado antes de cada passo.



O QUE MUDOU

A e B terminam no passo 2. No passo 3, E e F já estão nas vagas deles. Nenhuma vaga fica presa.

Analogia: ônibus de linha. Em cada ponto desce quem chegou e sobe quem estava esperando.

OS NÚMEROS

Passos até atender os 8	**15 (era 19)**
Casas com trabalho: 49 de 60	**82% (era 64%)**
Passo médio da resposta	**8,8 (era 15,5)**

Quem decide: o escalonador, antes de cada passo

Um laço pequeno que roda dezenas de vezes por segundo. Ele olha a fila e os blocos livres de memória.

1

Tira quem terminou

O pedido gerou o token de fim? Sai do lote e devolve os blocos de memória.

2

Chama da fila

Tem bloco livre e vaga no lote? Entra o próximo pedido que estava esperando.

3

Faltou memória? Pausa

Os pedidos crescem a cada token. Se a memória acabar, um pedido sai e volta depois.

4

Roda um passo

Um token para cada pedido do lote, com os pesos lidos uma vez só.

ANALOGIA

O porteiro de uma balada lotada: sai um, entra um. Se lotar, alguém espera lá fora.

NO VLLM

--max-num-seqs: quantos pedidos no lote. --max-num-batched-tokens: quantos tokens por passo.

4 · CHUNKED PREFILL

O problema: um prompt longo congela todo mundo

20 pedidos estão escrevendo, um token a cada 40 ms. Chega um pedido novo com um documento de 8.000 tokens para ler.

CADA BARRA É UM PASSO · A LARGURA É QUANTO TEMPO ELE LEVA · CINZA: PASSO NORMAL (40 MS)



POR QUE ESSE PASSO É TÃO LONGO

8.000 tokens × 16 bilhões de contas	**128 tri**
÷ 80 trilhões de contas por segundo	**1,6 s**
Um passo normal (só os 20 decodes)	**40 ms**

O QUE ACONTECE

Ler o prompt (prefill) é **um passo só**. Os 20 que estavam escrevendo esperam ele acabar: **40× um passo normal**.

Na tela deles, o texto para no meio da frase por 1,6 segundo.

ANALOGIA

Fila do supermercado: um carrinho cheio no caixa, e 20 pessoas com 1 item cada esperando ele passar tudo.

4 · CHUNKED PREFILL

A solução: ler o prompt em pedaços, junto com os outros

Cada passo tem um limite de 512 tokens. Primeiro entram os 20 que estão escrevendo; o resto do limite vai para um pedaço do documento.

MESMA ESCALA DE TEMPO · AGORA SÃO 17 PASSOS MÉDIOS EM VEZ DE 1 GIGANTE



DENTRO DE UM PASSO (512 TOKENS)



cinza: 1 token de cada um dos 20 que estão escrevendo

OS NÚMEROS

8.000 tokens ÷ 492 por passo	**17 passos**
512 tokens × 16 bi ÷ 80 tri/s	**103 ms**
Documento lido em	**1,7 s (era 1,6)**

PARA QUEM JÁ ESTAVA ESCREVENDO

Um token a cada **103 ms** por 1,7 s, em vez de travar 1,6 s. Ninguém congela.

BÔNUS

O documento usa as contas; os 20 decodes usam a memória. No mesmo passo, um completa o outro.

5 · PREFIX CACHING

Começo igual, fichas iguais: fazer uma vez e emprestar

Um token só olha para os tokens que vieram ANTES dele. Se vários pedidos começam igual, as fichas desse começo saem idênticas.

TRÊS PEDIDOS AO MESMO ASSISTENTE

Maria	Você é um assistente do PDC… (2.000 tokens)	Como instalo o Docker?
João	Você é um assistente do PDC… (2.000 tokens)	O que é quantização?
Pedro	Você é um assistente do PDC… (2.000 tokens)	Meu modelo não carrega.

verde: prompt de sistema, igual em todos · azul: a pergunta de cada um

NA MEMÓRIA

125 blocos do começo
2.000 ÷ 16 · 1 cópia para os 3

Maria | **João** | **Pedro**

blocos próprios: só a pergunta

COMO O SERVIDOR ACHA

Cada bloco cheio ganha uma impressão digital (hash) dos seus tokens e de tudo antes dele. Pedido novo: procura.

ANALOGIA

Uma apostila igual para a turma toda: imprime uma vez e todo mundo consulta. Cada um só escreve a sua resposta.

5 · PREFIX CACHING

Quanto economiza: 20 pedidos, um prompt de sistema

Llama-3.1-8B numa RTX 4090. Cada pedido: 2.000 tokens de instruções iguais + 100 tokens de pergunta.

SEM PREFIX CACHING	
20 × (2.000 + 100) tokens	**42.000**
Tempo de leitura (prefill)	**8,4 s**
Fichas: 42.000 × 128 KB	**5,5 GB**

COM PREFIX CACHING	
2.000 uma vez + 20 × 100	**4.000**
Tempo de leitura (prefill)	**0,8 s**
Fichas: 4.000 × 128 KB	**0,5 GB**

10,5× menos contas: o primeiro token de cada pessoa sai antes

10,5× menos memória de fichas: sobra espaço para mais pedidos no lote

Regra prática: o que é fixo vai no começo, o que muda vai no fim. Um token diferente no início (a data de hoje, o nome do usuário) muda a impressão digital de todos os blocos depois dele, e nada é reaproveitado.

NA PRÁTICA

Onde ficam esses botões nos runtimes

Cada linha é uma técnica desta apresentação. A semana 9 (Wevton) entra em cada runtime; confira o --help da sua versão.

Para que serve	vLLM	llama.cpp (llama-server)	Ollama
Quantos pedidos no lote	--max-num-seqs	--parallel N (-np): N vagas	OLLAMA_NUM_PARALLEL
Memória para as fichas	--gpu-memory-utilization (0,9)	-c: contexto total, dividido entre as vagas	contexto × pedidos paralelos
Batching contínuo	sempre ligado	ligado por padrão (--cont-batching)	herdado do llama.cpp
Chunked prefill	--max-num-batched-tokens	-b e -ub: tokens por passo	herdado do llama.cpp
Prefix caching	--enable-prefix-caching (padrão no V1)	cache_prompt por vaga; --cache-reuse	reaproveita o começo da conversa

Exemplo llama.cpp: -c 8192 --parallel 4 dá 4 vagas de 2.048 tokens cada. Mais vagas, conversas mais curtas.

RESUMO

As cinco peças, cada uma numa frase

PEÇA	PROBLEMA	SOLUÇÃO	ANALOGIA
Lote	O decode espera a memória	Pesos lidos uma vez servem 20 pedidos	*ônibus*
KV cache	Cada pedido guarda as suas fichas	É a memória delas que limita o lote	*fichas de leitura*
PagedAttention	Reserva do pior caso: 75% vazio	Blocos de 16 tokens, com índice	*fichário de folhas soltas*
Batching contínuo	Vagas presas até o mais longo acabar	O lote é remontado a cada passo	*ônibus de linha*
Chunked prefill	Prompt longo congela os outros	O prompt entra em pedaços	*caixa do supermercado*
Prefix caching	Começo igual é recalculado	Blocos iguais são compartilhados	*apostila da turma*

PARA MEXER E PARA LER

Material da keyword

LABORATÓRIO

KV cache e batching

As partes desta apresentação para mexer: tamanho do lote, quantos pedidos cabem na GPU, blocos do PagedAttention, estático × contínuo, chunked prefill e prefix caching.

`laboratorios/batching/`

CÓDIGO

batching.py

Mede lotes de 1 a 32 na sua máquina (PyTorch ou numpy), calcula o KV cache por modelo (--memoria) e testa um servidor de verdade (--servidor).

`na pasta desta keyword`

LEITURA

Os artigos

Kwon et al. 2023, PagedAttention (arXiv:2309.06180) · Yu et al. 2022, Orca (OSDI) · Agrawal et al. 2024, Sarathi-Serve (OSDI) · Ainslie et al. 2023, GQA (arXiv:2305.13245).

`docs/mapa-do-time.md`

Liga com: semana 6 (KV cache de um pedido) · semana 7 principal (roofline) · semana 9 (runtimes) · semana 10 (TTFT e TPOT)

A RESPOSTA

Por que servir 20 pedidos ao mesmo tempo custa muito menos que 20× servir um?

1 · O CARO É A VIAGEM DOS PESOS

Para escrever um token, o modelo inteiro sai da memória: 34 ms de viagem para 0,2 ms de contas. Num lote de 20, a viagem é uma só e serve aos 20.

2 · O QUE CRESCE É O KV CACHE

Cada pedido traz as suas fichas, e é a memória delas que limita quantos cabem. PagedAttention e prefix caching fazem caber mais.

3 · A AGENDA MANTÉM O LOTE CHEIO

Batching contínuo troca pedidos a cada passo, sem vaga parada. Chunked prefill corta prompts longos para ninguém congelar.

No meu notebook, 20 juntos custaram 13% de 20 separados. Numa GPU, ~6%. O lote divide a viagem dos pesos entre todos.

O preço: cada pedido fica um pouco mais lento, e cada um precisa do seu espaço de fichas na memória.